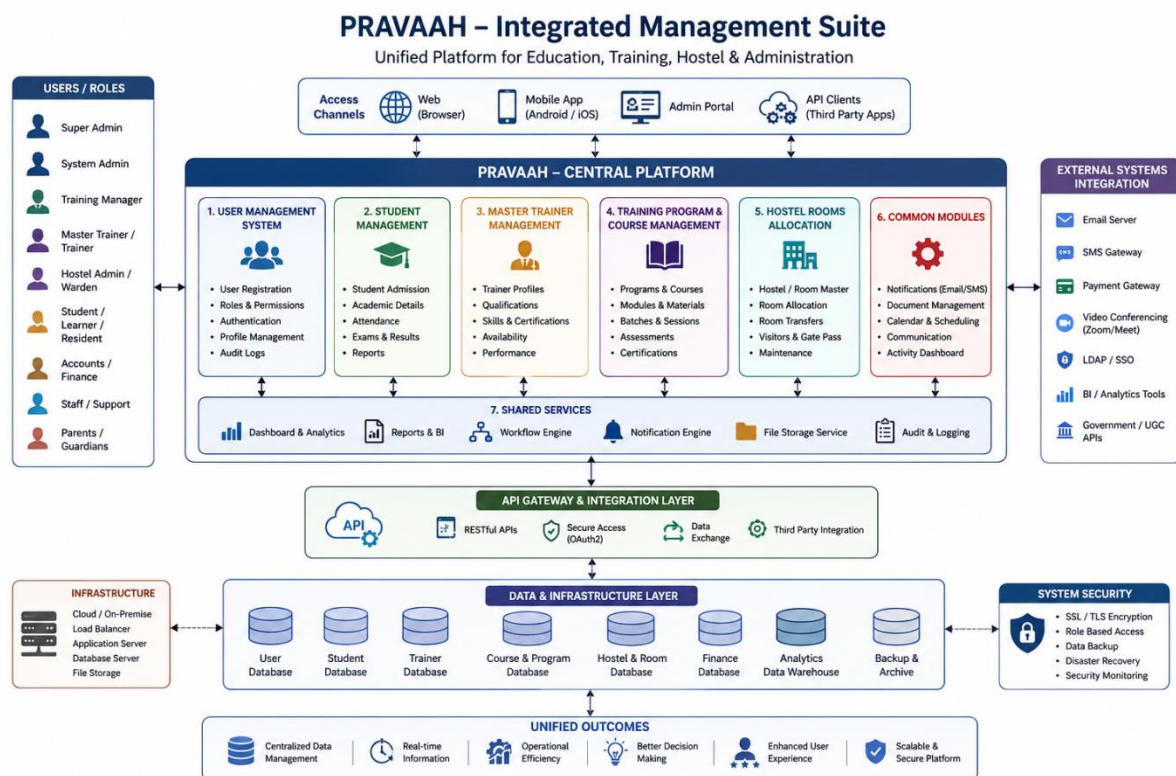


PRAVAAH – An integrated Management Suite

(An Unified Platform for Education, Training, Hostel & Administration)

Main Advantages of Having This System

1. Centralized Data Management
2. Real Time MIS Reports & Information Availability
3. Increased Operational Efficiency
4. Better Decision Making
5. Enhanced User Experience
6. Scalable & Secure Platform
7. Access Channels – Web App / Mobile App / Cloud App



Main Modules to be developed

1. User Management
2. Participants / Student Management
3. Master Trainer / Trainer Management
4. Training Program & Course Management
5. Hostel Rooms Allocation Management
6. Common Modules Management
7. Shared Services (Dashboard & Reports) Management

User Management

1. User Registration
2. Roles & Permissions Management
3. Authentication
4. Profile Management
5. Audit Logs Management
6. Reports

Student / Participants Management

1. Student Admission
2. Academic Details
3. Attendance
4. Exam & Results
5. Reports

Master Trainer Management

1. Trainer Profiles
2. Qualifications
3. Skills & Certifications
4. Availability

5. Performance
6. Reports

Training Program and Course Management

1. Programs & Courses
2. Modules & Materials
3. Batche & Sessions
4. Assessments
5. Certifications
6. Reports

Hostel Rooms Allocations

1. Hostel / Room Master
2. Room Allocation
3. Room Transfer
4. Visitors & Gate Pass
5. Maintainance
6. Reports

Common Modules

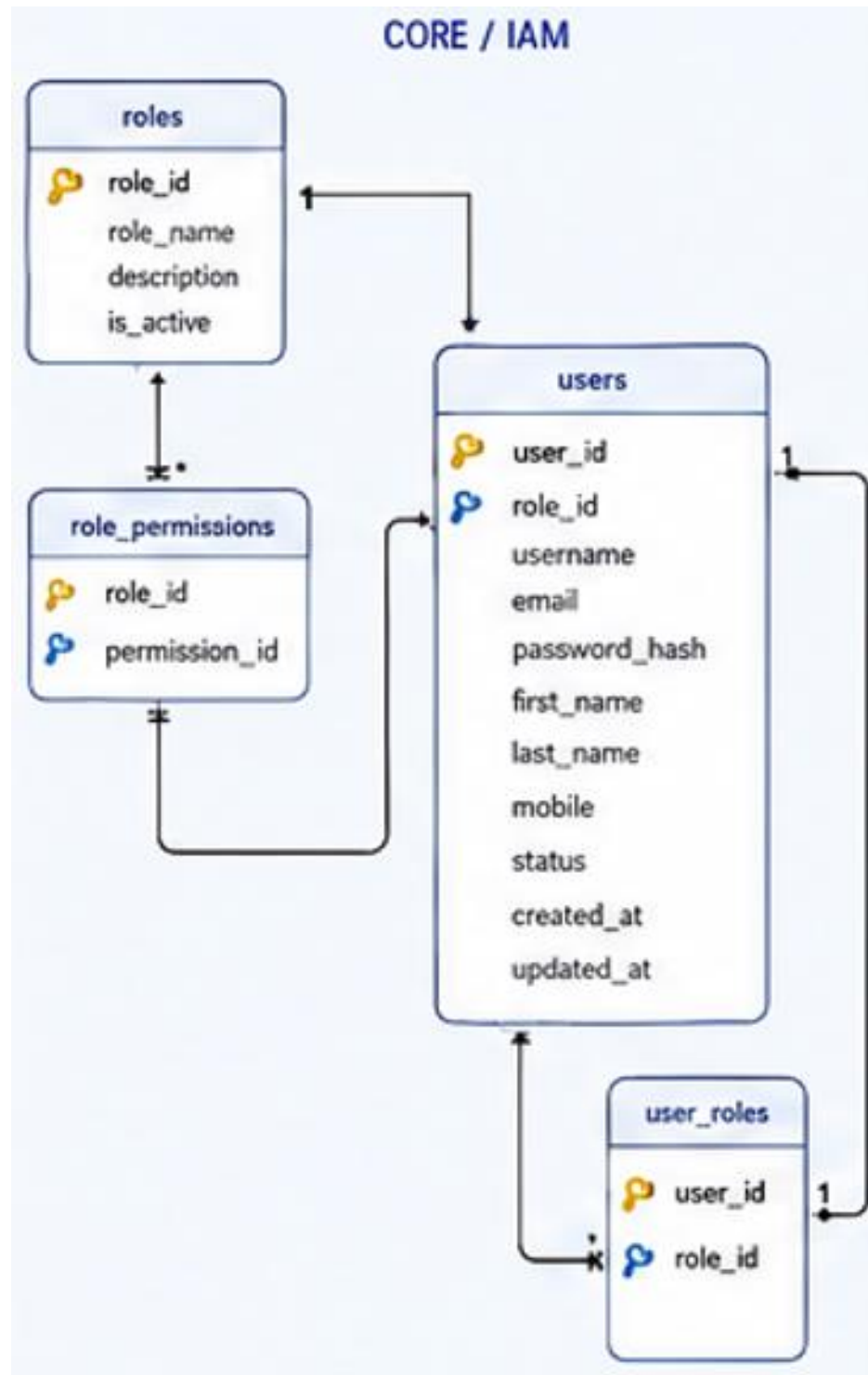
1. Notifications (Email / SMS / WhatsApp / Social Media)
2. Document Management
3. Calendar & Scheduling
4. Communication
5. Activity Dashboard

Shared Services

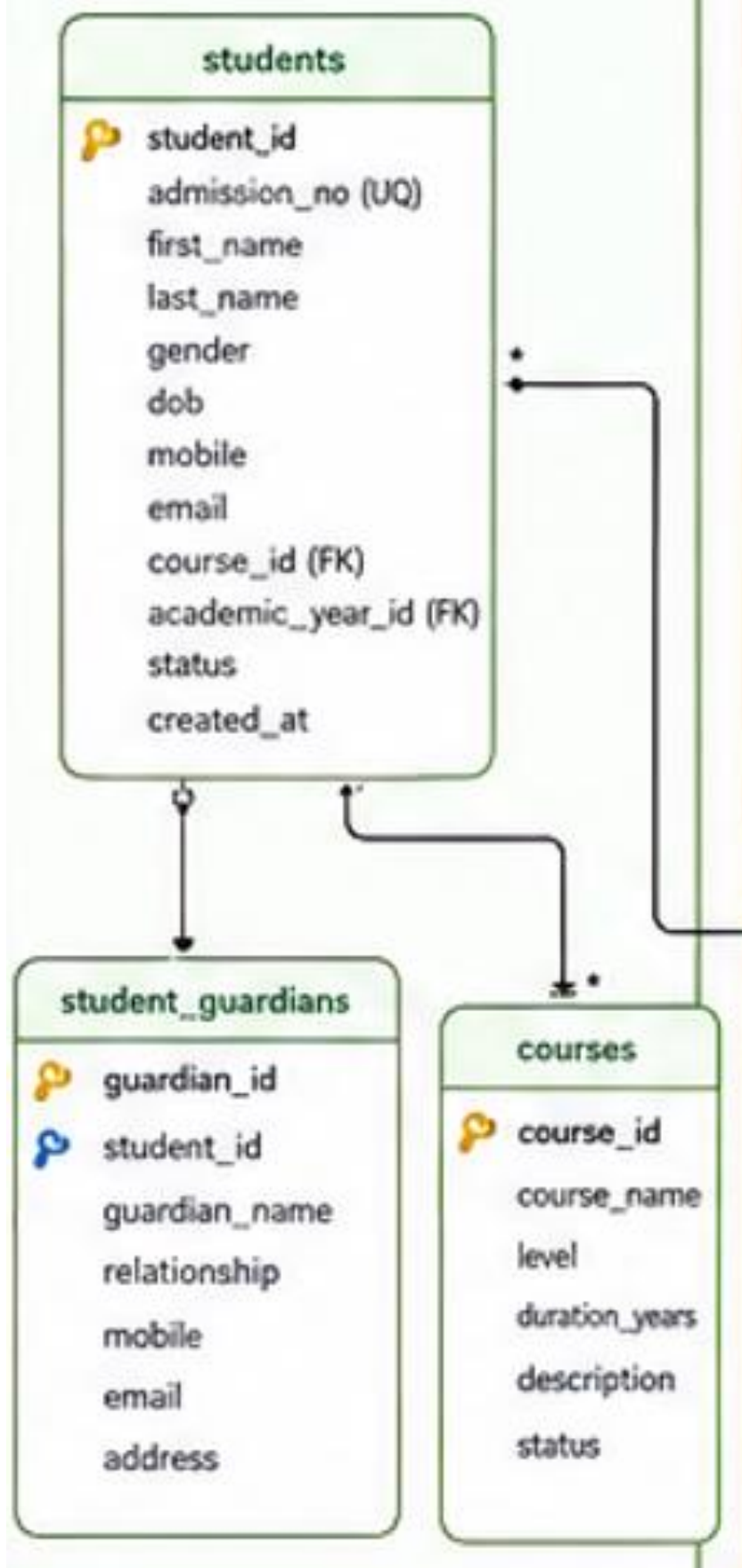
1. Dashboard & Analytics
2. Reports & Business Intelligence
3. Notification Engine
4. Workflow Engine
5. File Storage Service
6. Audit & Logging
7. Backup and Restore

Database – Pravaah

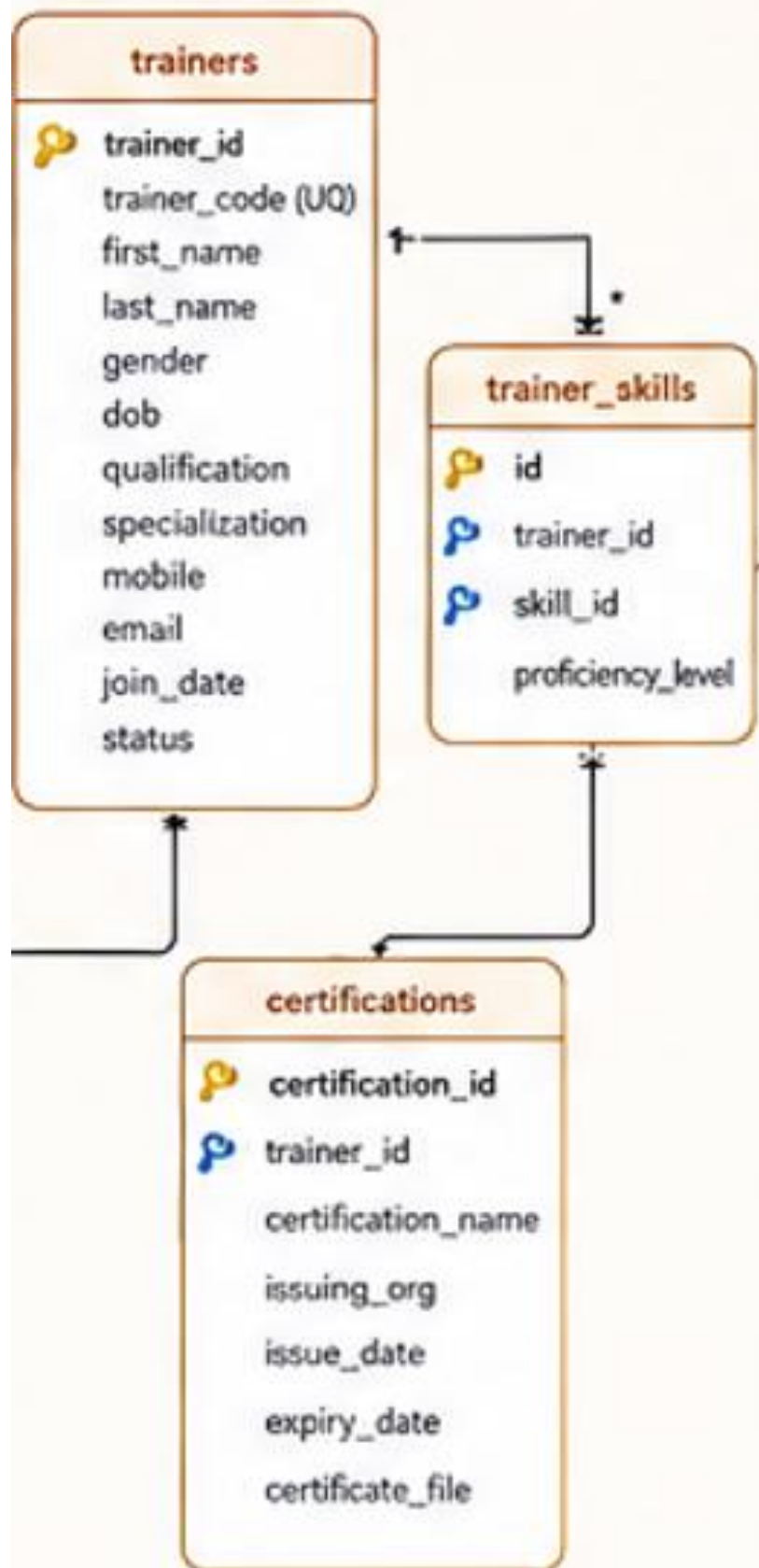
User Management



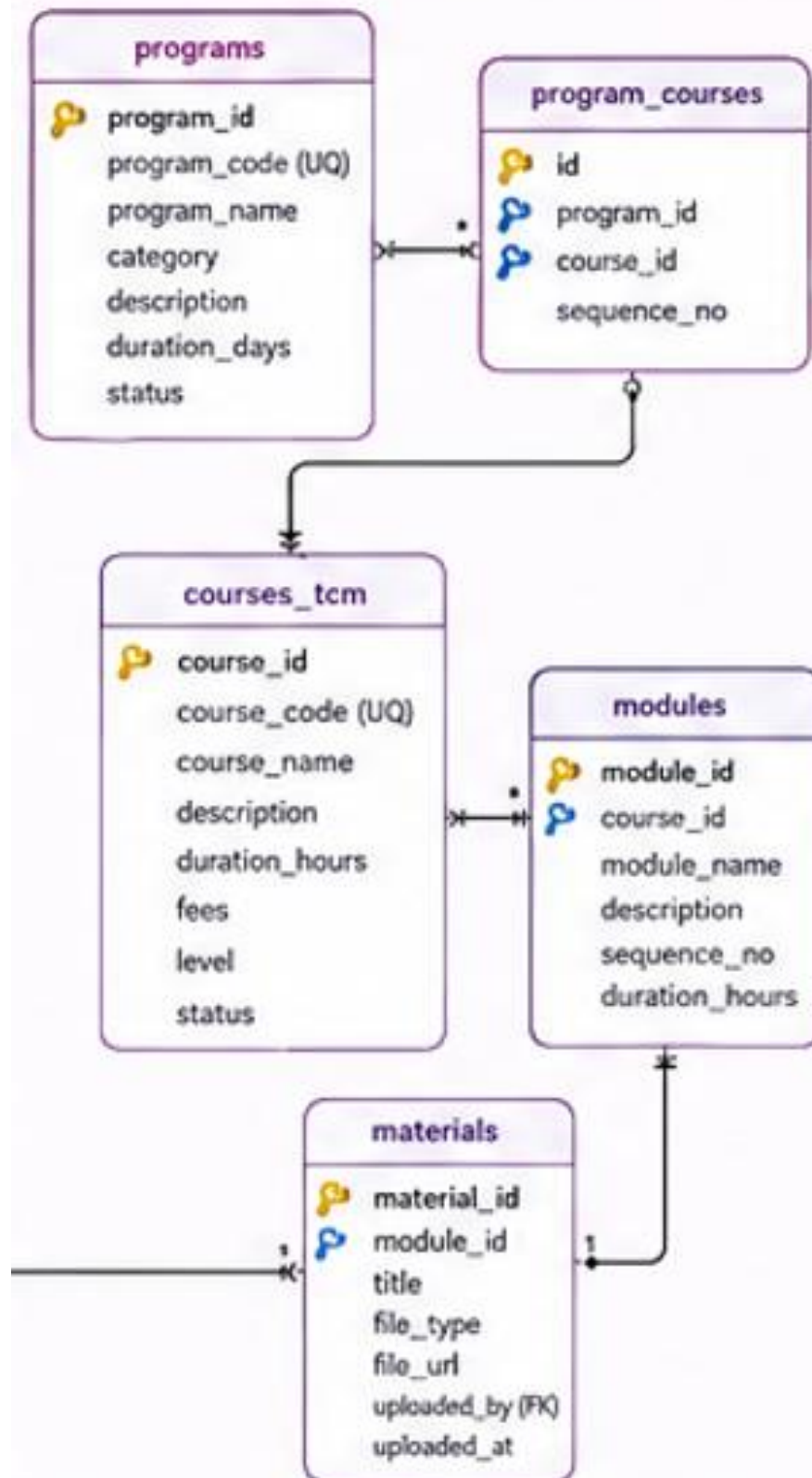
STUDENT MANAGEMENT



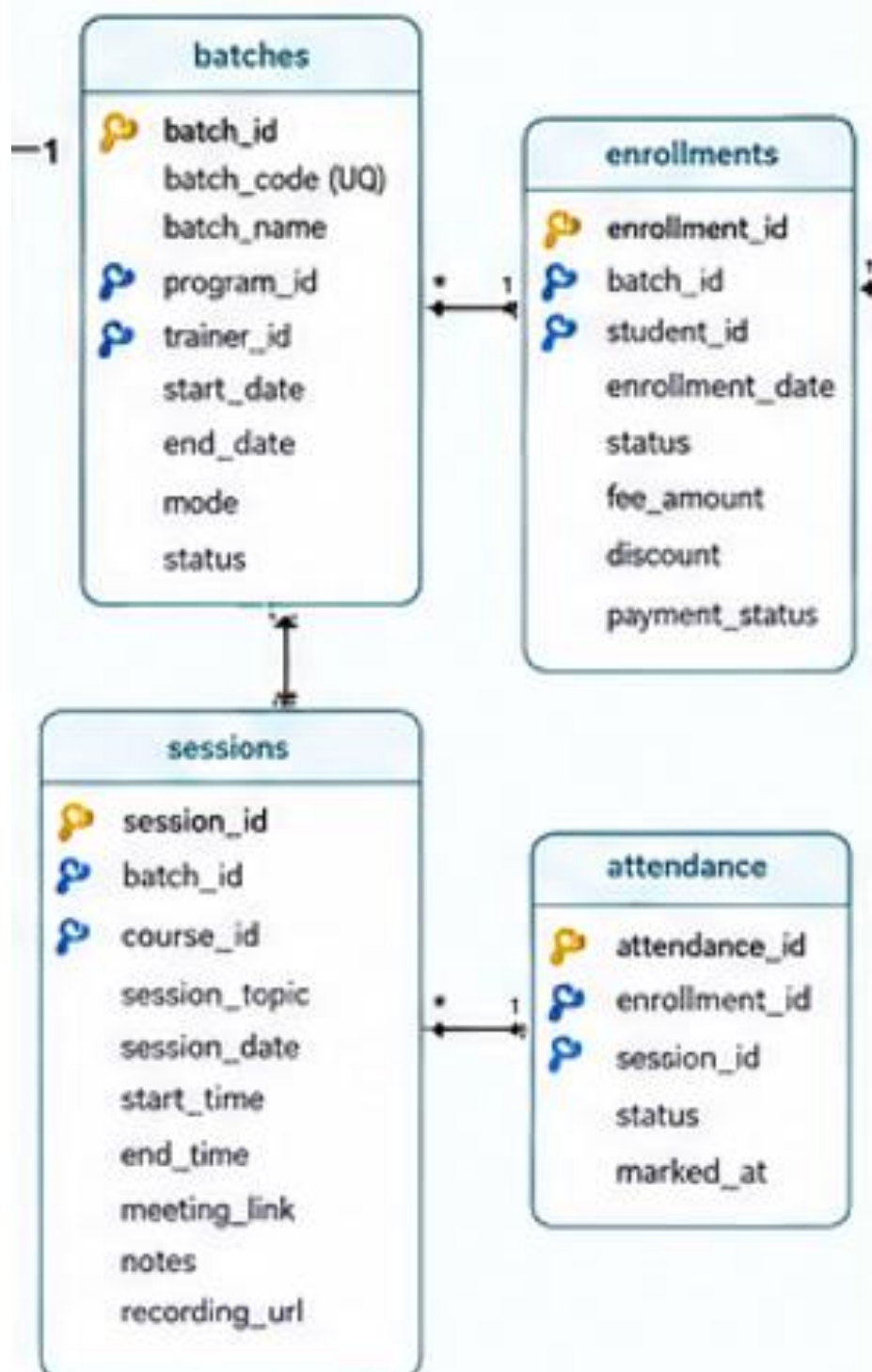
TRAINER MANAGEMENT



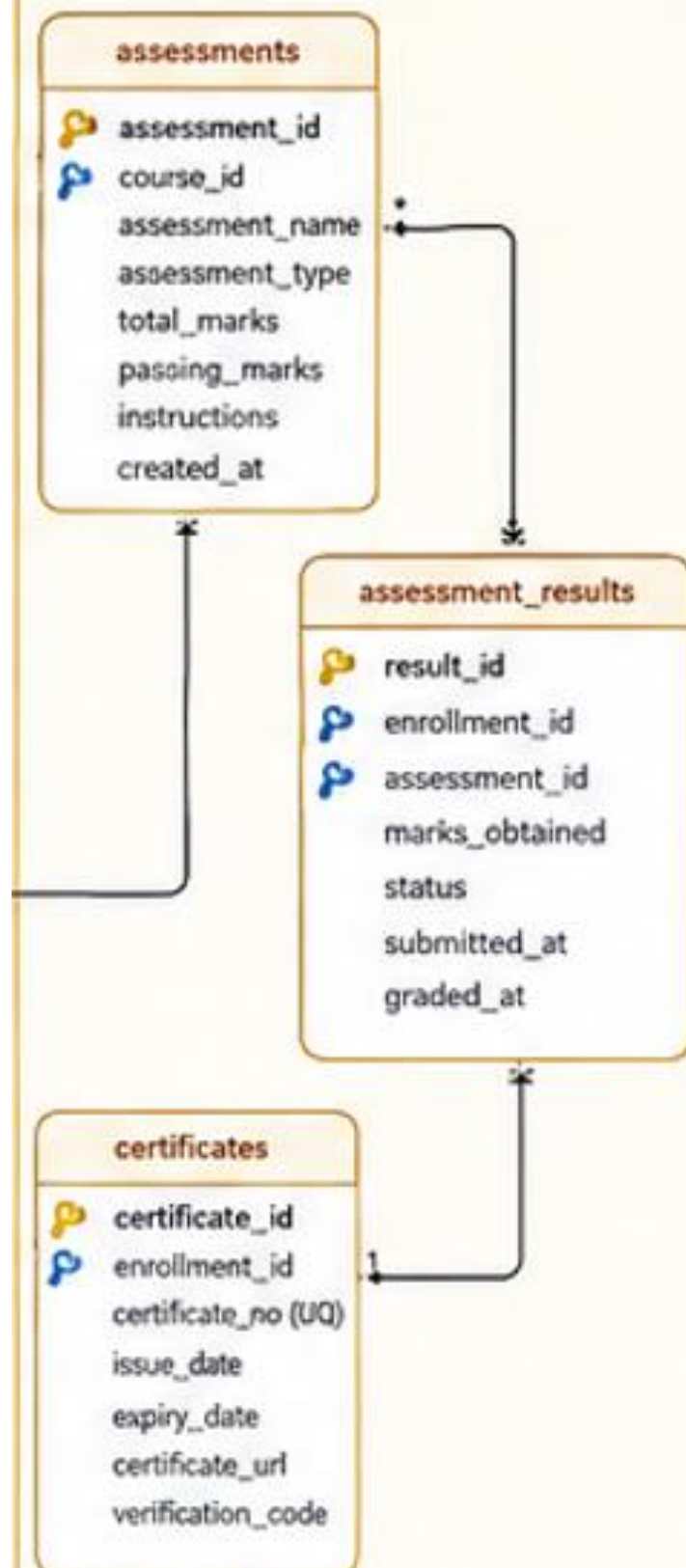
TRAINING PROGRAM & COURSE MANAGEMENT



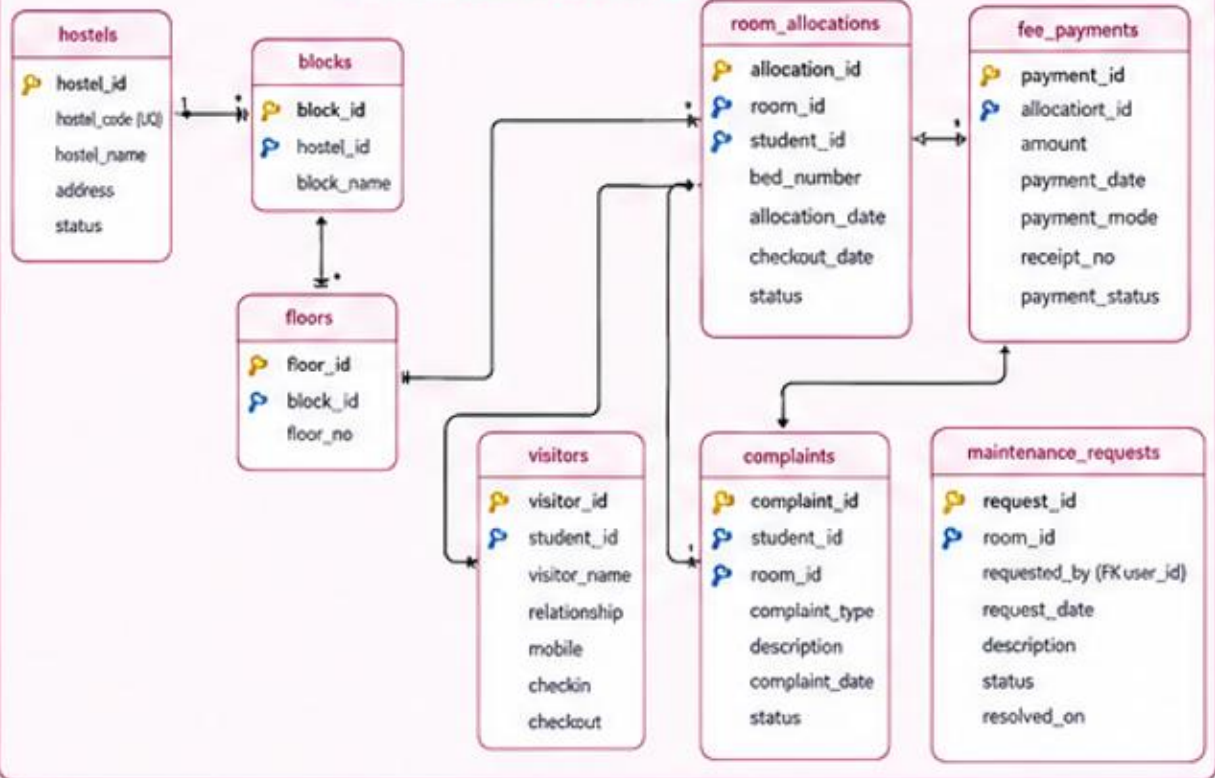
BATCH & SESSION MANAGEMENT



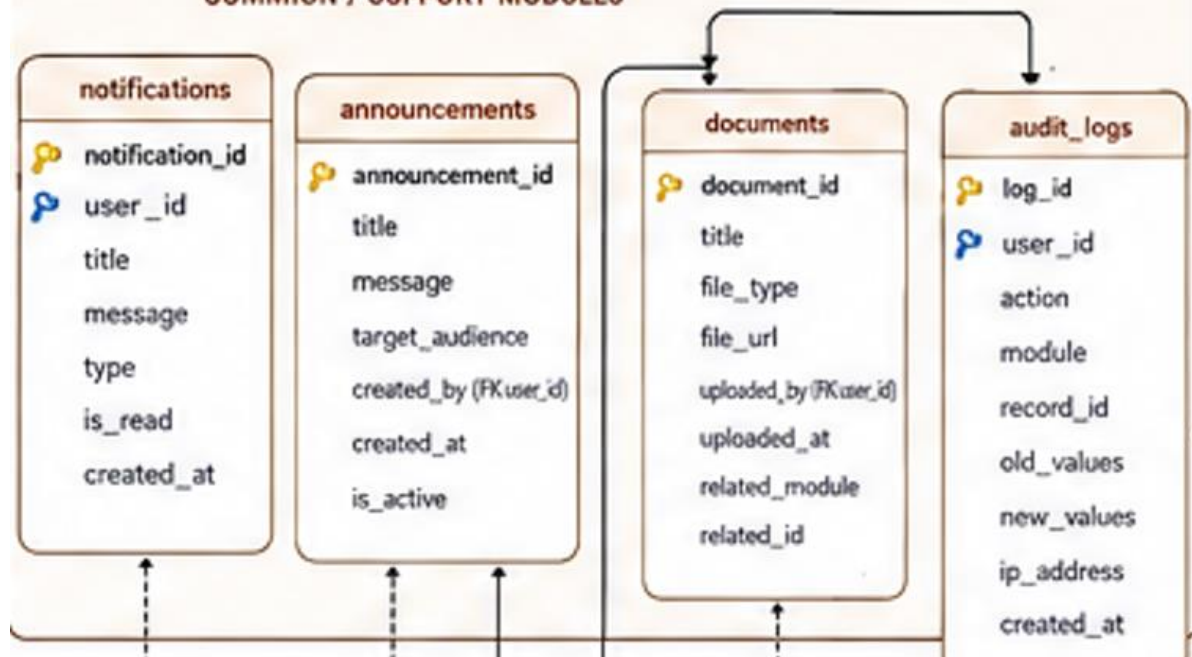
ASSESSMENT & CERTIFICATION



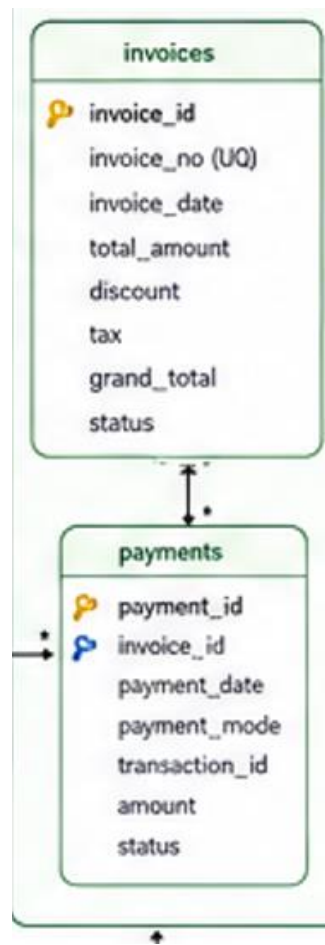
HOSTEL ROOMS ALLOCATION



COMMON / SUPPORT MODULES

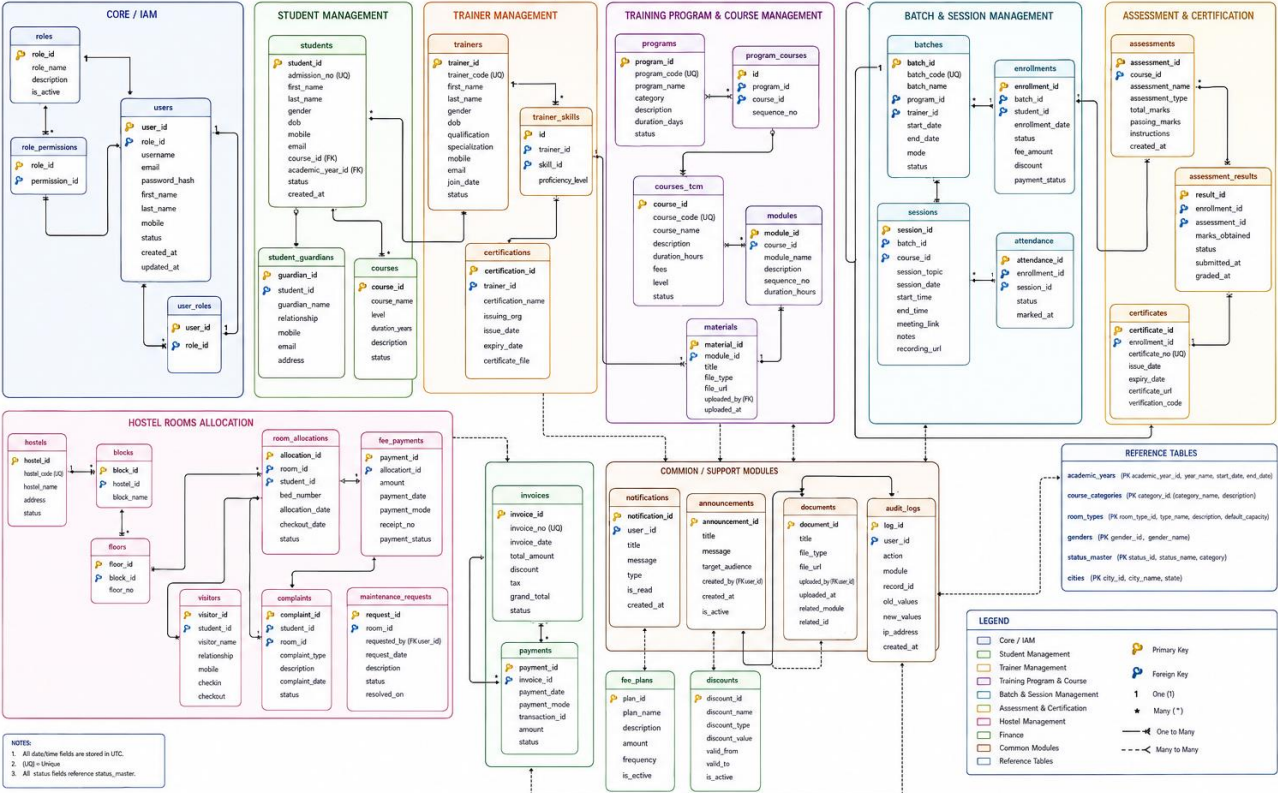


Accounts



PRAVAAH – Integrated Management Suite

ERD (Entity Relationship Diagram)



The ERD above represents a fairly large integrated platform (**PRAVAAH**) that combines:

- User Management
- Student Management
- Master Trainer Management
- Training Program & Courses
- Batch & Session Management
- Assessments & Certifications
- Hostel Management
- Finance
- Notifications
- Documents & Audit Logs

The full schema would contain following tables.

1. User & Authentication Module

```
CREATE TABLE roles(  
    role_id INT AUTO_INCREMENT PRIMARY KEY,  
    role_name VARCHAR(100) UNIQUE,  
    description TEXT,  
    is_active BOOLEAN DEFAULT TRUE  
);
```

Role_id	Role name	Description	Is_Active
1	Super Admin	Have All Rights	1
2	System Admin	Have system Access	1
3	Manager	Can Access Management Features	1
4	QA	Can Access QA Features	1

```
CREATE TABLE permissions(  
    permission_id INT AUTO_INCREMENT PRIMARY KEY,  
    permission_name VARCHAR(100) UNIQUE,  
    description TEXT  
);
```

Permission_id	Permission Name	Description
10	VIEW	Can View Only
20	MODIFY	Can View & Edit
30	AUTHORIZE	CAN AUTHORIZE OTHER USERS

```
CREATE TABLE users(  
    user_id INT AUTO_INCREMENT PRIMARY KEY,  
  
    username VARCHAR(100) UNIQUE,  
    email VARCHAR(150) UNIQUE,  
  
    password_hash VARCHAR(255),  
  
    first_name VARCHAR(100),  
    last_name VARCHAR(100),  
  
    mobile VARCHAR(20),  
  
    status ENUM(  
        'Active',  
        'Inactive',  
        'Blocked'  
    ) DEFAULT 'Active',  
  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

User_id	Username	Email	Password	Firstname	Lastname	Mobile	Status	Created_AT	Modified_AT

```

CREATE TABLE user_roles(
    id INT AUTO_INCREMENT PRIMARY KEY,

    user_id INT,
    role_id INT,

    FOREIGN KEY(user_id)
    REFERENCES users(user_id),

    FOREIGN KEY(role_id)
    REFERENCES roles(role_id)
);

```

Id	User_id	Role_id

```

CREATE TABLE role_permissions(
    id INT AUTO_INCREMENT PRIMARY KEY,

    role_id INT,
    permission_id INT,

    FOREIGN KEY(role_id)
    REFERENCES roles(role_id),

    FOREIGN KEY(permission_id)
    REFERENCES permissions(permission_id)
);

```

id	Role_id	Permission_id

2. Student Module

```
CREATE TABLE students(  
  
    student_id INT AUTO_INCREMENT PRIMARY KEY,  
  
    admission_no VARCHAR(30) UNIQUE,  
  
    first_name VARCHAR(100),  
    last_name VARCHAR(100),  
  
    gender VARCHAR(20),  
  
    dob DATE,  
  
    mobile VARCHAR(15),  
    email VARCHAR(100),  
  
    academic_year VARCHAR(20),  
  
    status VARCHAR(20),  
  
    created_at TIMESTAMP  
);
```

Student_id	Admission_no	First_name	Last_name	Gender	Dob	Mobile	Email	Academic year	Status	Created_at

```
CREATE TABLE student_guardians(  
  
    guardian_id INT AUTO_INCREMENT PRIMARY KEY,  
  
    student_id INT,  
  
    guardian_name VARCHAR(100),  
  
    relationship VARCHAR(50),  
  
    mobile VARCHAR(20),  
  
    email VARCHAR(100),  
  
    address TEXT,  
  
    FOREIGN KEY(student_id)  
    REFERENCES students(student_id)  
);
```

Guardian_id	Student_id	Guardian_name	Relationship	Mobile	Email	Address

3. Trainer Module

```
CREATE TABLE trainers(  
    trainer_id INT AUTO_INCREMENT PRIMARY KEY,  
    trainer_code VARCHAR(20) UNIQUE,  
    first_name VARCHAR(100),  
    last_name VARCHAR(100),  
    qualification VARCHAR(100),  
    specialization TEXT,  
    mobile VARCHAR(20),  
    email VARCHAR(100),  
    joining_date DATE,  
    status VARCHAR(20)  
);
```

Trainer id	Trainer code	First name	Last name	Qualification	Specialiation	Mobile	Email	Availability	Joiningdate

```
CREATE TABLE skills(  
    skill_id INT AUTO_INCREMENT PRIMARY KEY,  
    skill_name VARCHAR(100)  
);
```

Skill_id	Skill_name

```
CREATE TABLE trainer_skills(  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    trainer_id INT,  
    skill_id INT,  
    proficiency_level VARCHAR(50),  
    FOREIGN KEY(trainer_id)  
    REFERENCES trainers(trainer_id),  
    FOREIGN KEY(skill_id)  
    REFERENCES skills(skill_id)  
);
```

Id	Trainer_id	Skill_id	Proficiency_level

```
CREATE TABLE certifications(  
    certification_id INT AUTO_INCREMENT PRIMARY KEY,  
    trainer_id INT,  
    skill_id INT,  
    certification_date DATE,  
    status VARCHAR(20)
```

```
certification_id INT AUTO_INCREMENT PRIMARY KEY,  
  
trainer_id INT,  
  
certification_name VARCHAR(150),  
  
issuing_organization VARCHAR(150),  
  
issue_date DATE,  
  
expiry_date DATE,  
  
certificate_file VARCHAR(255),  
  
FOREIGN KEY(trainer_id)  
REFERENCES trainers(trainer_id)  
);
```

Certification id	Certification name	Certification organization	Issued date	Expiry date	Certification file

4. Course & Training Program Module

```
CREATE TABLE programs(  
    program_id INT AUTO_INCREMENT PRIMARY KEY,  
    program_code VARCHAR(30) UNIQUE,  
    program_name VARCHAR(200),  
    category VARCHAR(100),  
    description TEXT,  
    duration_days INT,  
    status VARCHAR(20)  
);
```


```
CREATE TABLE courses(  
    course_id INT AUTO_INCREMENT PRIMARY KEY,  
    course_code VARCHAR(30) UNIQUE,  
    course_name VARCHAR(200),  
    description TEXT,  
    duration_hours INT,  
    fees DECIMAL(10,2),  
    level VARCHAR(50)  
);
```


```
CREATE TABLE program_courses(  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    program_id INT,  
    course_id INT,  
    sequence_no INT,  
  
    FOREIGN KEY(program_id)  
    REFERENCES programs(program_id),  
  
    FOREIGN KEY(course_id)  
    REFERENCES courses(course_id)  
);
```

--	--	--	--


```

CREATE TABLE modules(

    module_id INT AUTO_INCREMENT PRIMARY KEY,

    course_id INT,

    module_name VARCHAR(200),

    description TEXT,

    sequence_no INT,

    duration_hours INT,

    FOREIGN KEY(course_id)
    REFERENCES courses(course_id)
);

```


```
CREATE TABLE materials(  
    material_id INT AUTO_INCREMENT PRIMARY KEY,  
    module_id INT,  
    title VARCHAR(200),  
    file_url VARCHAR(255),  
    uploaded_by INT,  
    uploaded_at DATETIME,  
    FOREIGN KEY(module_id)  
    REFERENCES modules(module_id),  
    FOREIGN KEY(uploaded_by)  
    REFERENCES users(user_id)  
);
```

5. Batch & Session Module

```
CREATE TABLE batches(  
    batch_id INT AUTO_INCREMENT PRIMARY KEY,  
    batch_code VARCHAR(20),  
    batch_name VARCHAR(100),  
    program_id INT,  
    trainer_id INT,  
    start_date DATE,  
    end_date DATE,  
    mode VARCHAR(30),  
    status VARCHAR(30),  
    FOREIGN KEY(program_id)  
    REFERENCES programs(program_id),  
    FOREIGN KEY(trainer_id)  
    REFERENCES trainers(trainer_id)  
);
```


```
CREATE TABLE enrollments(  
    enrollment_id INT AUTO_INCREMENT PRIMARY KEY,  
    batch_id INT,  
    student_id INT,  
    enrollment_date DATE,  
    status VARCHAR(20),  
    FOREIGN KEY(batch_id)  
    REFERENCES batches(batch_id),  
    FOREIGN KEY(student_id)  
    REFERENCES students(student_id)  
);
```


```
CREATE TABLE sessions(  
    session_id INT AUTO_INCREMENT PRIMARY KEY,  
    batch_id INT,
```

```

course_id INT,

session_topic VARCHAR(255),

session_date DATE,

start_time TIME,

end_time TIME,

meeting_link VARCHAR(255),

notes TEXT,

FOREIGN KEY(batch_id)
REFERENCES batches(batch_id),

FOREIGN KEY(course_id)
REFERENCES courses(course_id)
);

```


```

CREATE TABLE attendance(

attendance_id INT AUTO_INCREMENT PRIMARY KEY,

enrollment_id INT,

session_id INT,

status VARCHAR(20),

marked_at DATETIME,

FOREIGN KEY(enrollment_id)
REFERENCES enrollments(enrollment_id),

FOREIGN KEY(session_id)
REFERENCES sessions(session_id)
);

```

6. Assessment Module

```
CREATE TABLE assessments(  
    assessment_id INT AUTO_INCREMENT PRIMARY KEY,  
    course_id INT,  
    assessment_name VARCHAR(100),  
    total_marks INT,  
    passing_marks INT,  
    assessment_type VARCHAR(50),  
    FOREIGN KEY(course_id)  
    REFERENCES courses(course_id)  
);
```


```
CREATE TABLE assessment_results(  
    result_id INT AUTO_INCREMENT PRIMARY KEY,  
    enrollment_id INT,  
    assessment_id INT,  
    marks_obtained INT,  
    status VARCHAR(30),  
    FOREIGN KEY(enrollment_id)  
    REFERENCES enrollments(enrollment_id),  
    FOREIGN KEY(assessment_id)  
    REFERENCES assessments(assessment_id)  
);
```


```
CREATE TABLE certificates(  
    certificate_id INT AUTO_INCREMENT PRIMARY KEY,  
    enrollment_id INT,  
    certificate_no VARCHAR(100),  
    issue_date DATE,  
    certificate_url VARCHAR(255),  
    FOREIGN KEY(enrollment_id)  
    REFERENCES enrollments(enrollment_id)
```

) ;

7. Hostel Management Module

```
CREATE TABLE hostels(  
    hostel_id INT AUTO_INCREMENT PRIMARY KEY,  
    hostel_name VARCHAR(100),  
    address TEXT  
);
```


```
CREATE TABLE rooms(  
    room_id INT AUTO_INCREMENT PRIMARY KEY,  
    hostel_id INT,  
    room_no VARCHAR(20),  
    floor_no INT,  
    room_type VARCHAR(50),  
    capacity INT,  
    FOREIGN KEY(hostel_id)  
    REFERENCES hostels(hostel_id)  
);
```


```
CREATE TABLE room_allocations(  
    allocation_id INT AUTO_INCREMENT PRIMARY KEY,  
    student_id INT,  
    room_id INT,  
    allocation_date DATE,  
    checkout_date DATE,  
    FOREIGN KEY(student_id)  
    REFERENCES students(student_id),  
    FOREIGN KEY(room_id)  
    REFERENCES rooms(room_id)  
);
```

8. Finance Module

```
CREATE TABLE invoices(  
    invoice_id INT AUTO_INCREMENT PRIMARY KEY,  
    invoice_no VARCHAR(50),  
    total_amount DECIMAL(10,2),  
    tax DECIMAL(10,2),  
    grand_total DECIMAL(10,2),  
    status VARCHAR(20)  
);
```


```
CREATE TABLE payments(  
    payment_id INT AUTO_INCREMENT PRIMARY KEY,  
    invoice_id INT,  
    payment_date DATE,  
    payment_mode VARCHAR(50),  
    transaction_id VARCHAR(100),  
    amount DECIMAL(10,2),  
    FOREIGN KEY(invoice_id)  
    REFERENCES invoices(invoice_id)  
);
```

9. Common Modules

```
CREATE TABLE notifications(  
    notification_id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT,  
    title VARCHAR(200),  
    message TEXT,  
    created_at DATETIME,  
    FOREIGN KEY(user_id)  
    REFERENCES users(user_id)  
);
```

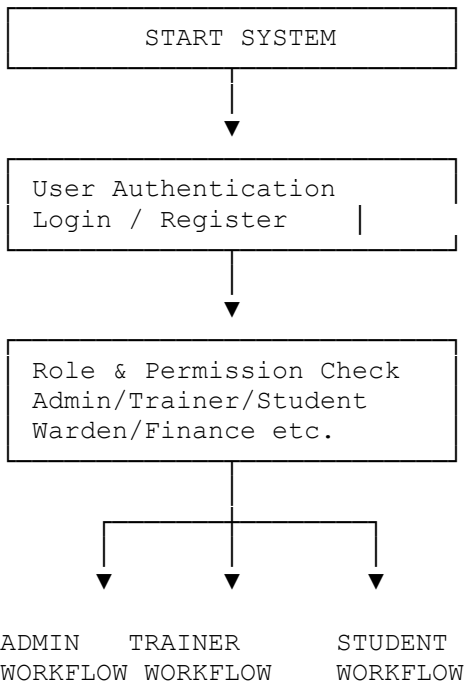

```
CREATE TABLE documents(  
    document_id INT AUTO_INCREMENT PRIMARY KEY,  
    title VARCHAR(200),  
    file_type VARCHAR(50),  
    file_url VARCHAR(255),  
    uploaded_by INT,  
    uploaded_at DATETIME,  
    FOREIGN KEY(uploaded_by)  
    REFERENCES users(user_id)  
);
```


```
CREATE TABLE audit_logs(  
    log_id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT,  
    action TEXT,  
    ip_address VARCHAR(100),  
    created_at DATETIME,  
    FOREIGN KEY(user_id)  
    REFERENCES users(user_id)  
);
```

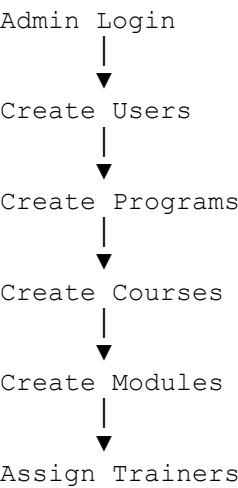

WORKFLOWS

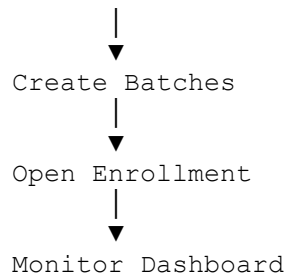
For **PRAVAAH – Integrated Management Suite**, the workflow should show how all modules interact in a single ecosystem rather than operating independently. Since the system combines User Management, Student Management, Trainer Management, Course Management, Hostel Allocation, Assessments, Finance, and Notifications, the workflow can be organized into operational stages.

PRAVAAH – System Workflow

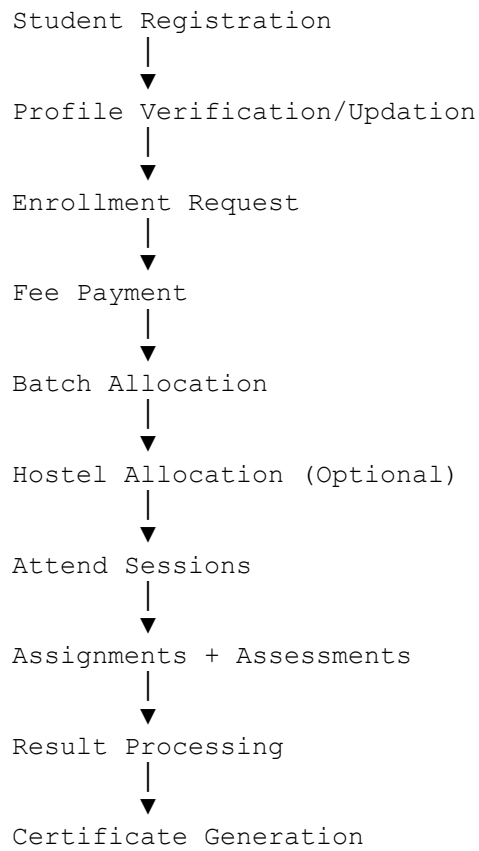


Admin Workflow

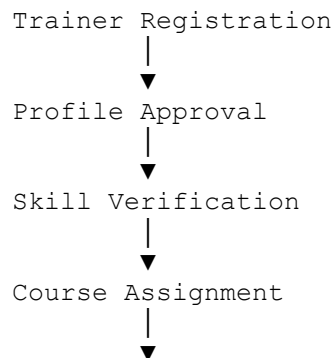


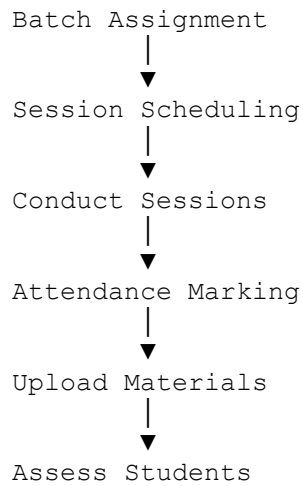


Student Workflow

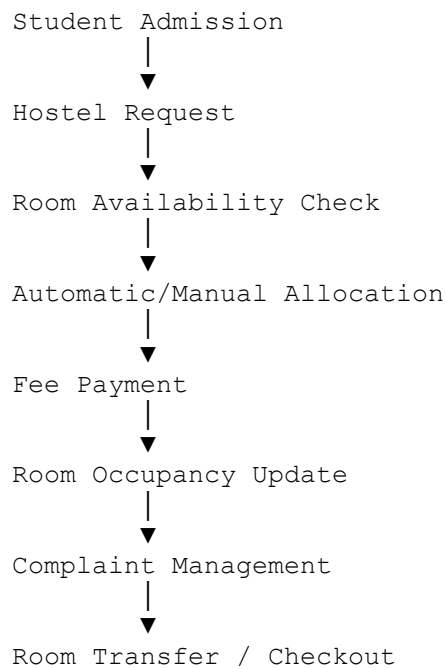


Trainer Workflow

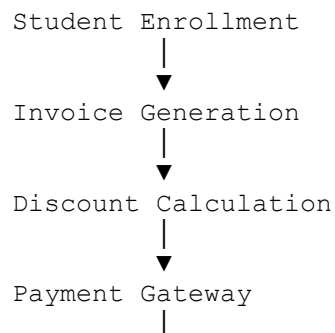


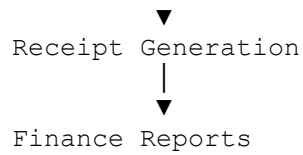


Hostel Workflow

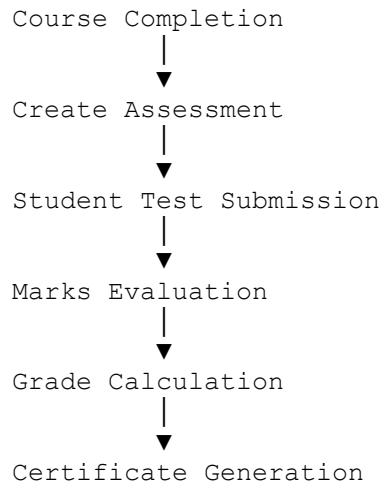


Finance Workflow

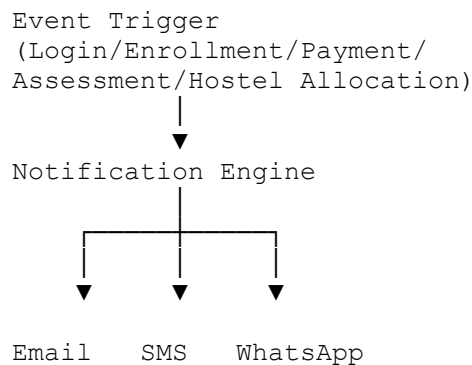




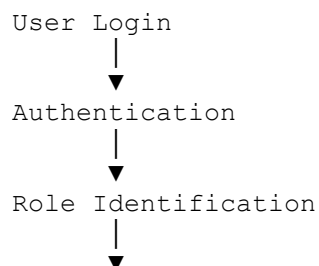
Assessment Workflow

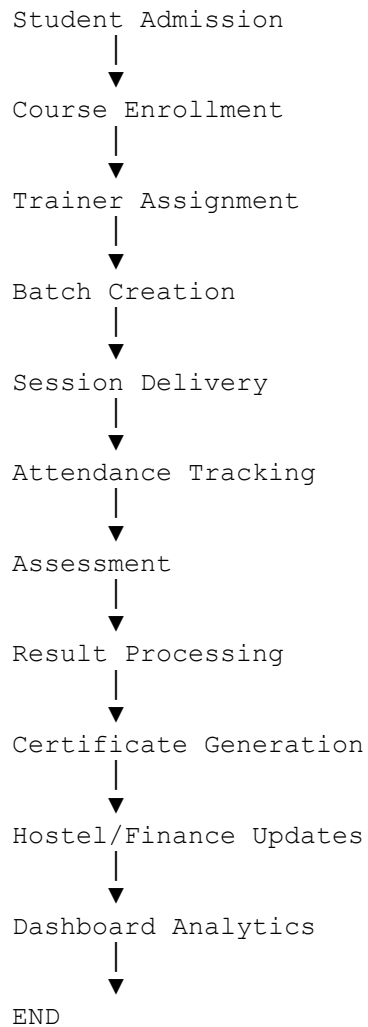


Notification Workflow



Integrated End-to-End Workflow





This workflow can directly be converted into Django apps and APIs:

Project – Pravaah

Apps ->

accounts/
students/
trainers/
courses/
batches/
sessions/
attendance/
hostels/
finance/
notifications/
certificates/
reports/
dashboard/
api/

For **PRAVAAH** as the main Django project with multiple integrated apps, create the project first and then add each module as a separate Django app. This gives clean separation and makes future maintenance easier.

Proposed structure:

```
PRAVAAH/  
├── accounts/  
├── users/  
├── students/  
├── trainers/  
├── programs/  
├── courses/  
├── batches/  
├── sessions/  
├── attendance/  
├── assessments/  
├── certificates/  
├── hostels/  
├── rooms/  
├── allocations/  
├── finance/  
├── notifications/  
├── reports/  
├── dashboard/  
├── api/  
├── media/  
├── static/  
└── manage.py
```

Step 1: Create Virtual Environment

Windows:

```
mkdir Pravaah  
cd Pravaah  
  
python -m venv env  
env\Scripts\activate
```

Linux/Mac:

```
mkdir Pravaah  
cd Pravaah  
  
python3 -m venv env  
source env/bin/activate
```

Step 2: Install Django and Required Packages

```
pip install django  
pip install mysqlclient  
pip install pillow  
pip install djangorestframework  
pip install django-crispy-forms
```

```
pip install django-filter
pip install django-allauth
pip install django-cors-headers
pip install django-import-export
pip install django-widget-tweaks
```

Optional packages:

```
pip install celery
pip install redis
pip install django-extensions
pip install drf-yasg
```

Step 3: Create Main Django Project

```
django-admin startproject pravaah .
```

The dot (.) creates the project in the current folder.

Folder becomes:

```
Pravaah/
env/
manage.py

pravaah/
    settings.py
    urls.py
    asgi.py
    wsgi.py
```

Step 4: Create All Django Apps

Run these one by one:

```
python manage.py startapp accounts
python manage.py startapp users
python manage.py startapp students
python manage.py startapp trainers
python manage.py startapp programs
python manage.py startapp courses
python manage.py startapp batches
python manage.py startapp sessions
python manage.py startapp attendance
python manage.py startapp assessments
```

```
python manage.py startapp certificates  
python manage.py startapp hostels  
python manage.py startapp rooms  
python manage.py startapp allocations  
python manage.py startapp finance  
python manage.py startapp notifications  
python manage.py startapp reports  
python manage.py startapp dashboard  
python manage.py startapp api
```

Step 5: Register Apps in `settings.py`

Open:

`pravaah/settings.py`

Modify:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
  
    'rest_framework',  
    'crispy_forms',  
    'corsheaders',  
  
    'accounts',  
    'users',  
    'students',  
    'trainers',  
    'programs',  
    'courses',  
    'batches',  
    'sessions',  
    'attendance',  
    'assessments',  
    'certificates',  
    'hostels',  
    'rooms',  
    'allocations',  
    'finance',  
    'notifications',  
    'reports',  
    'dashboard',  
    'api'
```

```
]
```

Step 6: Configure MySQL Database

Install MySQL if not already installed.

Edit:

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.mysql',  
        'NAME': 'pravaah_db',  
        'USER': 'root',  
        'PASSWORD': 'password',  
        'HOST': 'localhost',  
        'PORT': '3306',  
    }  
}
```

Create database:

```
CREATE DATABASE pravaah_db;
```

Step 7: Configure Static and Media Files

Add in settings.py:

```
STATIC_URL='static/'  
  
STATICFILES_DIRS=[  
    BASE_DIR/'static'  
]  
  
MEDIA_URL='/media/'  
  
MEDIA_ROOT=BASE_DIR/'media'
```

Create directories:

```
mkdir static  
mkdir media
```

Step 8: Create URLs for Apps

Example:

Create:

```
students/urls.py
from django.urls import path
from . import views

urlpatterns=[

    path('',views.index,name='students')
]
```

Main pravaah/urls.py

```
from django.contrib import admin
from django.urls import path,include

urlpatterns=[

path('admin/',admin.site.urls),

path('students/',include('students.urls')),
path('trainers/',include('trainers.urls')),
path('courses/',include('courses.urls')),
path('hostels/',include('hostels.urls')),
path('finance/',include('finance.urls')),
path('dashboard/',include('dashboard.urls')),

]
```

Step 9: Create Initial Migrations

```
python manage.py makemigrations
```

```
python manage.py migrate
```

Step 10: Create Superuser

```
python manage.py createsuperuser
```

Example:

```
Username: admin
```

```
Email: admin@gmail.com
```

```
Password: *****
```

Step 11: Run Server

```
python manage.py runserver
```

Open:

`http://127.0.0.1:8000`

Admin panel:

`http://127.0.0.1:8000/admin`

Step 12: Create Custom User Model (recommended before migrations)

Create:

```
users/models.py
from django.contrib.auth.models import AbstractUser
from django.db import models

class User(AbstractUser):

    mobile=models.CharField(
        max_length=15,
        blank=True
    )

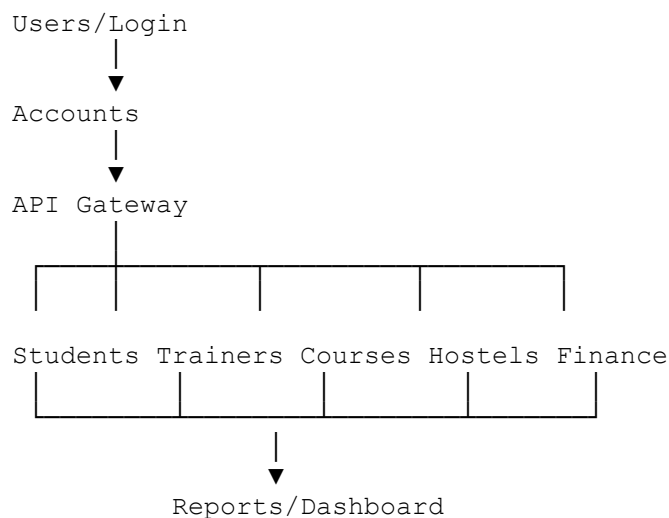
    profile_image=models.ImageField(
        upload_to='profiles/',
        blank=True
    )

    role=models.CharField(
        max_length=50
    )
```

Add to settings.py

```
AUTH_USER_MODEL='users.User'
```

Final PRAVAAH Architecture

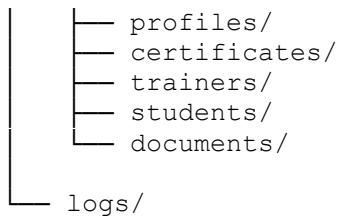


This creates the foundation for a large-scale integrated Django ERP-like system for PRAVAAH.

You have listed the Django apps for PRAVAAH. A cleaner enterprise structure is to separate shared functionality and domain modules so they scale better.

Recommended structure:

```
PRAVAAH/
├── manage.py
├── requirements.txt
├── env/
├── pravaah/
│   ├── settings.py
│   ├── urls.py
│   ├── wsgi.py
│   └── asgi.py
├── apps/
│   ├── accounts/
│   │   ├── models.py
│   │   ├── views.py
│   │   ├── urls.py
│   │   ├── admin.py
│   │   ├── forms.py
│   │   ├── serializers.py
│   │   ├── permissions.py
│   │   └── services.py
│   ├── students/
│   ├── trainers/
│   ├── courses/
│   ├── batches/
│   ├── sessions/
│   ├── attendance/
│   ├── hostels/
│   ├── finance/
│   ├── notifications/
│   ├── certificates/
│   ├── reports/
│   ├── dashboard/
│   └── api/
├── templates/
│   ├── accounts/
│   ├── students/
│   ├── trainers/
│   ├── courses/
│   ├── batches/
│   ├── hostels/
│   └── dashboard/
├── static/
│   ├── css/
│   ├── js/
│   ├── images/
│   └── fonts/
└── media/
```



Create this structure from the terminal:

```
mkdir Pravaah
cd Pravaah

python -m venv env

env\Scripts\activate
```

Install dependencies:

```
pip install django
pip install djangorestframework
pip install mysqlclient
pip install pillow
pip install django-allauth
pip install django-cors-headers
```

Create the project:

```
django-admin startproject pravaah .
```

Create the apps folder:

```
mkdir apps
cd apps
```

Create apps:

```
python ../manage.py startapp accounts
python ../manage.py startapp students
python ../manage.py startapp trainers
python ../manage.py startapp courses
python ../manage.py startapp batches
python ../manage.py startapp sessions
python ../manage.py startapp attendance
python ../manage.py startapp hostels
python ../manage.py startapp finance
python ../manage.py startapp notifications
python ../manage.py startapp certificates
```

```
python ../manage.py startapp reports

python ../manage.py startapp dashboard

python ../manage.py startapp api
```

Return to root:

```
cd ..
```

Add to settings.py:

```
INSTALLED_APPS=[

    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    'rest_framework',
    'corsheaders',

    'apps.accounts',
    'apps.students',
    'apps.trainers',
    'apps.courses',
    'apps.batches',
    'apps.sessions',
    'apps.attendance',
    'apps.hostels',
    'apps.finance',
    'apps.notifications',
    'apps.certificates',
    'apps.reports',
    'apps.dashboard',
    'apps.api',
]
```

Create app URLs:

Example (students/urls.py):

```
from django.urls import path
from . import views

urlpatterns=[

    path('',views.home,name='students')
]
```

Update main urls.py:

```
from django.urls import path,include

urlpatterns=[
```

```
path(
    'students/',
    include('apps.students.urls')
),

path(
    'trainers/',
    include('apps.trainers.urls')
),

path(
    'courses/',
    include('apps.courses.urls')
),

path(
    'hostels/',
    include('apps.hostels.urls')
),

path(
    'dashboard/',
    include('apps.dashboard.urls')
),

]
```

Run migrations:

```
python manage.py makemigrations
```

```
python manage.py migrate
```

```
python manage.py createsuperuser
```

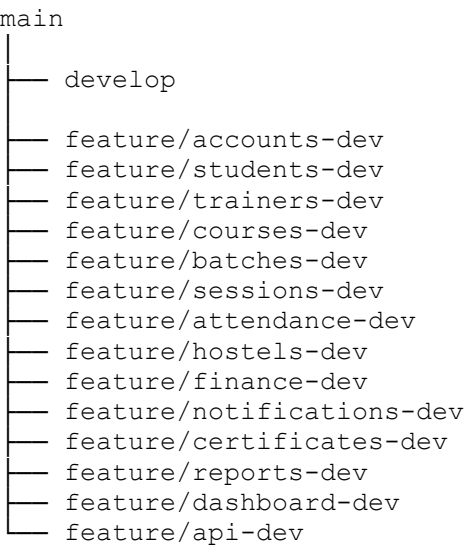
```
python manage.py runserver
```

This structure keeps PRAVAAH modular and makes it easier to add future apps such as placements/, library/, inventory/, transport/, or franchise/ without restructuring the project.

For PRAVAAH, the cleanest approach is to use **one central Git repository** and assign **each Django app to a different developer/team**, while maintaining a shared integration branch. This is a common workflow used in ERP and enterprise systems.

Architecture:

PRAVAAH (Main Repository)



Developer assignment example:

Developer	Assigned App
Developer 1	accounts
Developer 2	students
Developer 3	trainers
Developer 4	courses
Developer 5	batches
Developer 6	sessions
Developer 7	attendance
Developer 8	hostels
Developer 9	finance
Developer 10	notifications
Developer 11	certificates

Developer	Assigned App
-----------	--------------

Developer 12	reports/dashboard
--------------	-------------------

Developer 13	API integration
--------------	-----------------

Step 1: Create a GitHub repository

Create a repository named:

Pravaah

Initialize locally:

```
git init
```

Connect to GitHub:

```
git remote add origin https://github.com/yourusername/Pravaah.git
```

Check:

```
git remote -v
```

Step 2: Create .gitignore

For Django:

```
__pycache__/  
*.pyc  
env/  
venv/  
  
db.sqlite3  
  
media/  
  
*.log  
  
.idea/  
.vscode/  
  
staticfiles/  
  
.env
```

Commit initial setup:

```
git add .  
git commit -m "Initial Pravaah project setup"  
git push -u origin main
```

Step 3: Create development branches

Main branches:

```
git checkout -b develop
```

```
git push -u origin develop
```

Create module branches:

```
git checkout develop
```

```
git checkout -b feature/accounts-dev
```

```
git checkout -b feature/students-dev
```

```
git checkout -b feature/trainers-dev
```

```
git checkout -b feature/courses-dev
```

```
git checkout -b feature/batches-dev
```

```
git checkout -b feature/sessions-dev
```

```
git checkout -b feature/attendance-dev
```

```
git checkout -b feature/hostels-dev
```

```
git checkout -b feature/finance-dev
```

```
git checkout -b feature/notifications-dev
```

```
git checkout -b feature/certificates-dev
```

```
git checkout -b feature/reports-dev
```

```
git checkout -b feature/dashboard-dev
```

```
git checkout -b feature/api-dev
```

Push all:

```
git push --all
```

Step 4: Add developers to GitHub

Open your repository:

Use:

Settings → Collaborators → Add people

Invite each developer using their GitHub username.

Assign ownership:

- Student team → students
- Finance team → finance
- Hostel team → hostels
- API team → api

Step 5: Developer workflow

Example for Student module developer:

Clone repository:

```
git clone https://github.com/yourusername/Pravaah.git
```

Move into project:

```
cd Pravaah
```

Switch to assigned branch:

```
git checkout feature/students-dev
```

Work on:

```
apps/students/
```

Commit changes:

```
git add .  
git commit -m "Added student models and views"  
git push
```

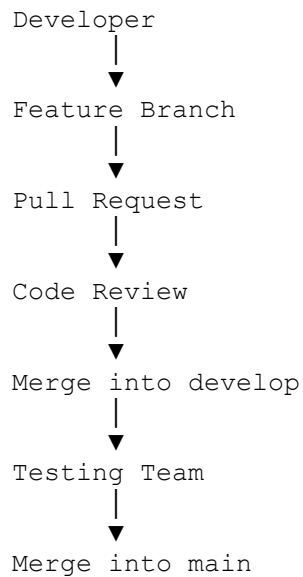
Step 6: Pull Request workflow

Developer creates:

```
feature/students-dev  
  |  
  ▼  
Pull Request  
  |  
  ▼  
develop branch  
  |  
  ▼  
Testing  
  |
```

main ▼

Typical sequence:



Step 7: Resolve integration conflicts

Before pushing changes:

```
git checkout feature/students-dev
git pull origin develop
```

Resolve conflicts if they exist:

```
git add .
git commit -m "Resolved merge conflicts"
git push
```

Step 8: Suggested ownership rules for PRAVAAH

Shared files that only lead developers should modify:

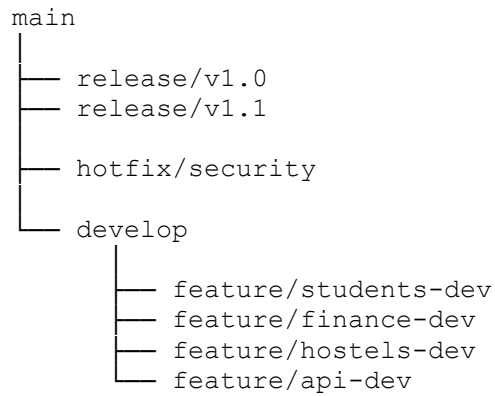
```
pravaah/settings.py
pravaah/urls.py
requirements.txt
Dockerfile
.env
```

Module developers modify only:

```
apps/students/*
```

```
apps/finance/*
apps/hostels/*
apps/trainers/*
```

Step 9: Recommended production workflow

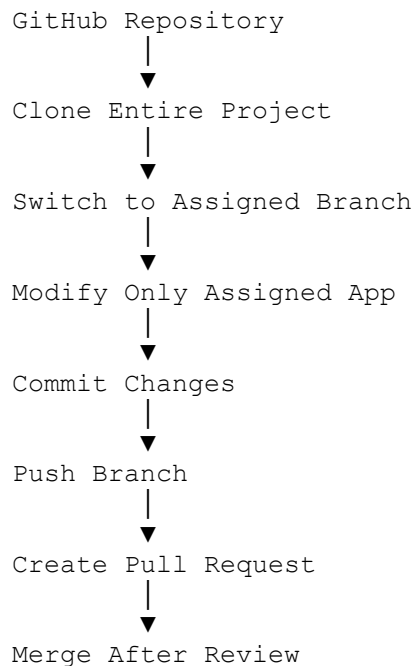


This follows a GitFlow-style model and allows 10–20 developers to work simultaneously on PRAVAAH without overwriting each other’s work while still integrating into a single Django application.

To avoid overlapping work in a multi-developer Django project like **PRAVAAH**, developers generally should **not download ZIP files, edit them, and upload the whole project again**. That approach often overwrites others' changes.

Instead, use a Git workflow where each developer changes only their own branch and merges through pull requests.

Typical flow:



First-time setup

Step 1: Clone the project

Copy repository URL from GitHub:

```
git clone https://github.com/yourusername/Pravaah.git
```

Move into project:

```
cd Pravaah
```

Step 2: Create virtual environment

Windows:

```
python -m venv env
env\Scripts\activate
```

Linux/Mac:

```
python3 -m venv env
```

```
source env/bin/activate
```

Install dependencies:

```
pip install -r requirements.txt
```

Step 3: Download latest code before starting work

Every day before editing:

```
git checkout develop
git pull origin develop
```

This updates your local project with everyone else's changes.

Work on only your assigned app

Example: Student developer

Switch to your branch:

```
git checkout feature/students-dev
```

Edit only:

```
apps/students/
```

Avoid changing:

```
pravaah/settings.py
pravaah/urls.py
requirements.txt
```

unless specifically assigned.

Check what changed before committing

View modified files:

```
git status
```

Example output:

```
modified: apps/students/models.py
modified: apps/students/views.py
```

Review changes:

```
git diff
```

Upload only your changes

Add files:

```
git add .
```

Commit:

```
git commit -m "Added student admission module"
```

Push:

```
git push origin feature/students-dev
```

Only your branch is updated.

Merge latest team changes before continuing

If other developers changed code:

```
git checkout develop
git pull origin develop

git checkout feature/students-dev
git merge develop
```

If there are no conflicts:

```
Merge successful
```

Push:

```
git push
```

If a conflict happens

Git will show something like:

```
<<<<<<< HEAD
class Student(models.Model):
    mobile=models.CharField(max_length=15)
=====
class Student(models.Model):
    phone=models.CharField(max_length=20)
```

```
>>>>>> develop
```

Choose the final version:

```
class Student(models.Model):  
    mobile=models.CharField(max_length=15)
```

Then:

```
git add .  
git commit -m "Resolved merge conflict"  
git push
```

Using GitHub Pull Requests

After pushing:

1. Open GitHub repository
2. Click **Pull Requests**
3. Click **New Pull Request**
4. Select:

```
feature/students-dev  
    ↓  
develop
```

5. Add description
6. Submit

The lead developer reviews and merges.

If you receive a ZIP instead of a Git repository

Extract:

```
Pravaah.zip  
    ↓  
Extract  
    ↓  
Pravaah/
```

Initialize Git:

```
git init
```

Connect to GitHub:

```
git remote add origin https://github.com/yourusername/Pravaah.git
```

Push:

```
git add .  
git commit -m "Initial upload"  
git push -u origin main
```

After that, stop exchanging ZIP files and use Git branches.

For PRAVAAH with many apps and developers, a useful rule is:

```
One repository  
One app ownership  
One feature branch per developer  
Pull requests before merging  
Never upload the entire project repeatedly
```

That prevents one developer from accidentally replacing someone else's work.

A complete list of *all* Git commands is very large (hundreds of commands and options). For practical development, especially for a Django project like **PRAVAAH**, developers typically rely on a core set of commands repeatedly. Below is a developer-oriented guide with **command → meaning → usage → effect on the project → real situation**.

Repository Setup Commands

Command	Meaning	Usage	Effect on Project	Typical Situation
<code>git init</code>	Initialize Git repository	Creates local repository tracking	Creates <code>.git</code> folder	Starting a new project
<code>git clone URL</code>	Copy existing repository	Downloads project from GitHub	Creates local working copy	Joining a team project
<code>git remote add origin URL</code>	Connect local repo to GitHub	Adds remote repository	Links project to GitHub	First-time GitHub setup
<code>git remote -v</code>	View remotes	Shows linked repositories	No code changes	Verify repository URL
<code>git remote remove origin</code>	Remove remote	Disconnects GitHub link	Removes remote mapping	Wrong repository linked

Example:

```
git init
git remote add origin https://github.com/company/Pravaah.git
```

Checking Project Status

Command	Meaning	Usage	Effect	Situation
<code>git status</code>	Show current changes	Displays modified files	No changes	Before commit
<code>git log</code>	Show commit history	Displays commits	No changes	Review history
<code>git log --oneline</code>	Compact history	Short commit view	No changes	Quick review
<code>git diff</code>	Show modifications	Compare changes	No changes	Check edits
<code>git show</code>	View commit details	Inspect commit	No changes	Debugging

Example:

```
git status
git diff
```

Developer scenario:

"I edited students/models.py and want to see what changed before committing."

Staging Commands

Command	Meaning	Effect	Situation
<code>git add file.py</code>	Add specific file	Moves file to staging area	One file changed
<code>git add .</code>	Add all changes	Stages everything	Multiple files changed
<code>git reset file.py</code>	Remove from staging	Unstage file	Added wrong file
<code>git restore --staged file.py</code>	Remove staged file	Unstage only	Accidental staging

Example:

```
git add students/models.py
```

Scenario:

Developer changed only one model and wants to commit only that file.

Commit Commands

Command	Meaning	Effect	Situation
<code>git commit -m "message"</code>	Save changes	Creates snapshot	Regular development
<code>git commit --amend</code>	Modify last commit	Updates previous commit	Wrong message
<code>git commit -a</code>	Commit tracked files	Saves changes automatically	Quick update

Example:

```
git commit -m "Added hostel allocation model"
```

Effect:

Creates a permanent project checkpoint

Branch Commands

Command	Meaning	Effect	Situation
<code>git branch</code>	List branches	No code changes	Check branches
<code>git branch branchname</code>	Create branch	Creates new branch	New feature
<code>git checkout branchname</code>	Switch branch	Changes working area	Work on module
<code>git checkout -b branchname</code>	Create + switch	Creates and moves	Start new feature
<code>git branch -d branchname</code>	Delete branch	Removes branch	Feature completed

Example:

```
git checkout -b feature/students-dev
```

Scenario:

Student module developer starts work.

Pull / Push Commands

Command	Meaning	Effect	Situation
<code>git pull origin main</code>	Download changes	Updates local project	Get team updates
<code>git fetch</code>	Download without merge	Gets remote info	Review before merge
<code>git push origin main</code>	Upload changes	Updates GitHub	Share code
<code>git push --all</code>	Push all branches	Uploads branches	Initial setup

Example:

```
git pull origin develop
```

Scenario:

Before starting work each morning.

Merge Commands

Command	Meaning	Effect	Situation
<code>git merge branchname</code>	Merge branches	Combines code	Integrate feature
<code>git merge --abort</code>	Cancel merge	Reverts merge process	Conflict issue

Example:

```
git merge develop
```

Scenario:

Student developer wants latest project updates.

Undo / Recovery Commands

Command	Meaning	Effect	Situation
<code>git restore file.py</code>	Discard changes	Returns file to previous state	Mistake
<code>git reset --soft HEAD~1</code>	Undo commit, keep changes	Moves commit back	Wrong commit
<code>git reset --hard HEAD~1</code>	Remove commit and changes	Deletes work	Dangerous rollback
<code>git revert commitID</code>	Reverse commit	Creates undo commit	Production fix

Example:

```
git restore students/views.py
```

Scenario:

Developer broke code accidentally.

Stash Commands

Command	Meaning	Effect	Situation
<code>git stash</code>	Temporarily save work	Removes changes from workspace	Switch tasks
<code>git stash list</code>	Show stashes	No code changes	Check saved work
<code>git stash apply</code>	Restore stash	Gets work back	Resume task
<code>git stash pop</code>	Restore and remove stash	Continues work	Resume development

Example:

```
git stash
```

Scenario:

```
Urgent bug appears while unfinished work exists.
```

Tag Commands

Command	Meaning	Effect	Situation
<code>git tag</code>	Show tags	No changes	View versions
<code>git tag v1.0</code>	Create version	Creates release marker	Release
<code>git push origin v1.0</code>	Upload tag	Publishes release	Deployment

Scenario:

```
PRAVAAH version 1.0 released.
```

GitHub CLI Commands (optional)

Command	Meaning	Situation
gh auth login	Login GitHub CLI	First setup
gh repo create	Create repository	New project
gh pr create	Create pull request	Submit feature
gh pr list	View pull requests	Review work
gh issue list	View issues	Track tasks

Example:

```
gh pr create
```

Daily workflow used by developers

```
git checkout develop
git pull origin develop

git checkout feature/students-dev

# make changes

git status

git add .

git commit -m "Added student registration"

git push origin feature/students-dev
```

Typical team situation:

Morning:
Pull latest changes

During day:
Create/edit code

Before lunch:
Commit local work

End of day:
Push branch

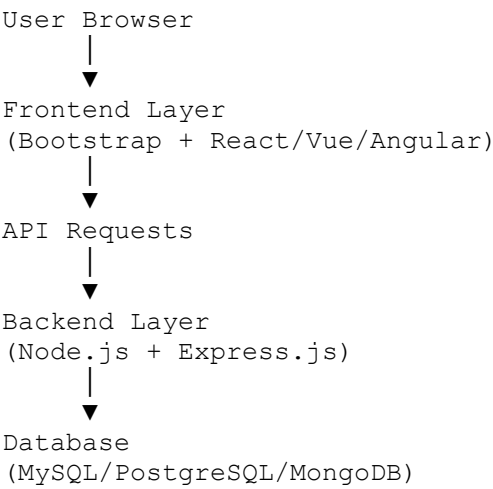
After testing:
Create Pull Request

After approval:
Merge into develop

This covers the commands most developers use in real projects roughly 90–95% of the time.

These technologies are often mentioned together, but they play **very different roles** in a full-stack application. Some are UI frameworks, some are frontend libraries, some run backend servers, and some provide APIs.

A simple picture first:



Side-by-side comparison

Technology	Type	Main Role	Runs Where	Purpose
Bootstrap	CSS Framework	UI styling	Browser	Makes pages responsive and beautiful
Vue.js	Frontend Framework	User interface	Browser	Build interactive frontend
React.js	Frontend Library	User interface	Browser	Build reusable UI components
Angular	Frontend Framework	Complete frontend solution	Browser	Build large enterprise applications
Node.js	Runtime Environment	Execute JavaScript outside browser	Server	Run backend code
Express.js	Backend Framework	API and routing	Server	Build web applications and APIs

1. Bootstrap

Role: **Design and styling**

Bootstrap gives ready-made components:

- Buttons
- Forms
- Cards
- Tables
- Navigation bars
- Modals
- Grid system

Without Bootstrap:

```
<button style="
background:blue;
padding:10px;
border-radius:5px;
">
Save
</button>
```

With Bootstrap:

```
<button class="btn btn-primary">
Save
</button>
```

Role in full stack:

Backend → Data
Frontend → React/Vue/Angular
Bootstrap → Appearance

Best for:

- Fast UI development
 - Responsive websites
 - Admin dashboards
-

2. Vue.js

Role: **Frontend framework**

Vue creates dynamic pages.

Example:

```
<div id="app">

{{message}}
```

```
</div>
new Vue({

  el:"#app",

  data:{

    message:"Hello"
  }

})
```

Features:

- Easy learning curve
- Two-way binding
- Component-based
- Lightweight

Role in full stack:

```
Backend API
  |
  ▼
Vue frontend
  |
  ▼
User interface
```

Best for:

- Small to medium applications
- Dashboards
- Rapid development

3. React.js

Role: **Frontend library**

React builds applications using reusable components.

Example:

```
function Welcome() {

  return (

    <h1>Hello World</h1>

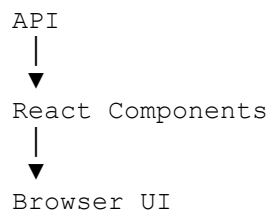
  )

}
```

Features:

- Virtual DOM
- Reusable components
- Fast rendering
- Huge ecosystem

Role in full stack:



Best for:

- Large applications
- Mobile apps via React Native
- Interactive interfaces

Examples:

- Facebook
- Instagram
- Netflix

React

4. Angular

Role: **Complete frontend framework**

Angular includes:

- Routing
- Forms
- Dependency Injection
- HTTP services
- Validation
- State management

Example:

```
@Component ({  
  selector: 'app-home',
```

```
template: '<h1>Hello</h1>'
}))

export class HomeComponent{}
```

Features:

- Uses TypeScript
- Strong structure
- Enterprise-ready

Role in full stack:

```
Backend API
  |
  ▼
Angular Application
  |
  ▼
Browser
```

Best for:

- Large enterprise systems
- ERP systems
- Banking software

Examples:

- Large corporate dashboards

Angular

5. Node.js

Role: JavaScript runtime

JavaScript normally runs in browsers.

Node.js allows JavaScript to run on servers.

Example:

```
console.log("Server Running")
```

Role in full stack:

Browser
|
Request
|
Node.js Server
|
Database

Features:

- Non-blocking
- Event-driven
- Fast I/O operations

Best for:

- APIs
- Chat systems
- Real-time applications

Examples:

- Chat applications
- Streaming services

Node.js

6. Express.js

Role: **Backend web framework for Node.js**

Node gives runtime capability.

Express gives structure:

- Routes
- Middleware
- APIs
- Authentication

Without Express:

```
http.createServer(...)
```

With Express:

```
const express=require('express')
```

```
const app=express()
```

```
app.get('/students', (req, res) => {  
  
  res.send("Students Data")  
  
})
```

Role in full stack:

```
Frontend  
  |  
API Request  
  |  
Express  
  |  
Database
```

Best for:

- REST APIs
- Authentication systems
- Backend services

Express.js

Full-stack example: PRAVAAH

If you build PRAVAAH using these technologies:

```
Frontend  
  
React.js  
Bootstrap  
  
  |  
  ▼  
  
REST API  
  
Express.js  
  
  |  
  ▼  
  
Node.js Runtime  
  
  |  
  ▼  
  
MySQL Database
```

or with Django:

Frontend

Bootstrap

React.js



Django REST API



MySQL

Exact differences in one line

- **Bootstrap** → Makes pages look good.
- **Vue.js** → Builds interactive frontend UI.
- **React.js** → Builds reusable UI components.
- **Angular** → Complete frontend framework for large systems.
- **Node.js** → Runs JavaScript on the server.
- **Express.js** → Creates APIs and routes on Node.js.

A common confusion is:

Node.js ≠ Express.js

React ≠ Angular ≠ Vue

Bootstrap ≠ React

because they solve different problems and are often used together rather than replacing one another.

